

คู่มือและขั้นตอนการเทรนโมเดล TinyML ตรวจจับการล้ม

(TinyML Fall Detection Model Training Guide)

กระบวนการประมวลผลข้อมูล Hybrid Dataset, การสกัด 18 คุณลักษณะ, การเทรนโครงข่ายประสาทเทียม และการแปลงเป็น C Header บน ESP32

1. บทนำและวัตถุประสงค์ (Introduction & Objective)

การตรวจจับการล้มบนอุปกรณ์ฮาร์ดแวร์ฝังตัว (Edge AI / TinyML)

เป็นแนวทางที่ตอบโจทย์ความปลอดภัยและความเป็นส่วนตัวสูงสุด เนื่องจากข้อมูลสัญญาณความเร่งจากเซ็นเซอร์ IMU (MPU-6050) จะถูกประมวลผลและตัดสินใจได้ทันทีบนไมโครคอนโทรลเลอร์ ESP32 โดยไม่ต้องพึ่งพาสัญญาณอินเทอร์เน็ตตลอดเวลา

แนวคิดการออกแบบ: เพื่อแก้ปัญหาด้านการแจ้งเตือนผิดพลาด (False Positive) จากการเคลื่อนไหวในชีวิตประจำวัน เช่น การนั่งเร็ว การก้มตัว หรือการล้มตัวลงนอน การเทรนโมเดลจึงใช้ **ชุดข้อมูลแบบผสมผสาน (Hybrid Dataset)** ระหว่างข้อมูลการล้มจริงจากอุปกรณ์และชุดข้อมูลกิจกรรมประจำวันมาตรฐานสากล

2. การเตรียมสภาพแวดล้อมและไลบรารี (Environment Setup)

ก่อนเริ่มการเทรนโมเดล ต้องเตรียมระบบคอมพิวเตอร์ที่ติดตั้ง Python 3.9 ขึ้นไป

และติดตั้งไลบรารีสำหรับการประมวลผลตัวเลขและ Machine Learning:

คำสั่งติดตั้งไลบรารี: รันคำสั่งติดตั้งแพ็คเกจผ่าน Terminal หรือ PowerShell:

```
pip install numpy pandas scikit-learn
```

2.1 โครงสร้างโฟลเดอร์ชุดข้อมูล (Datasets Structure)

- 1. โฟลเดอร์ข้อมูลการล้มจริง (Fall Data): D:\ssPro\CollectData_Web_Dataset\sensor_records*.csv (ข้อมูลจริงจากการทดสอบล้ม 52 ไฟล์)
- 2. โฟลเดอร์ข้อมูลกิจกรรมปกติ (SisFall ADL): D:\ssPro\SisFall_Dataset**\D*.txt (ข้อมูลกิจวัตรประจำวัน 2,707 ไฟล์ จาก 38 ผู้ทดสอบ)

- 3. สคริปต์การเทรนโมเดลหลัก: C:\Users\Earth\Desktop\Pro\Hardware1\tinyml\train_model.py

3. การประมวลผลสัญญาณและการจัดเตรียมข้อมูล (Signal Preprocessing)

- 1. การแปลงหน่วยทางกายภาพ (Unit Normalization): แปลงค่าความเร่ง ADXL345 ($\pm 16g$, 13-bit) เป็นหน่วย m/s^2 และแปลงความเร็วเชิงมุม ITG3200 ($\pm 2000 \text{ deg/s}$) เป็น rad/s เพื่อให้ตรงกับมาตรฐานหน่วยของ MPU-6050
- 2. การปรับความถี่สุ่ม (Downsampling 200Hz -> 50Hz): ข้อมูล SisFall ถูกบันทึกที่ความถี่ 200 Hz จึงทำการ Downsampling ลง 4 เท่า (`data[::4]`) ให้เหลือความถี่ 50 Hz เท่ากับความถี่สุ่มของฮาร์ดแวร์ ESP32
- 3. การตัดหน้าต่างข้อมูล (Sliding Window & Impact Window): กำหนดหน้าต่างข้อมูลขนาด 50 แซมเปิล (เท่ากับช่วงเวลา 1.0 วินาที ที่ความถี่ 50 Hz) โดยใช้วิธี Impact-centered Window ตัดช่วงเวลา 20 จุดก่อนจุดกระทบ และ 30 จุดหลังจุดกระทบ เพื่อให้ได้พฤติกรรมการล้มที่สมบูรณ์ที่สุด

4. การสกัดคุณลักษณะทางสถิติ 18 ค่า (Feature Engineering)

ในแต่ละหน้าต่างข้อมูล (50 แซมเปิล $\times$ 6 แกน) จะถูกสกัดเป็นเวกเตอร์คุณลักษณะทางสถิติจำนวน 18 มิติ เพื่อลดขนาดข้อมูลและเพิ่มความสามารถในการแยกแยะของโมเดล:

ลำดับ (#)	ชื่อคุณลักษณะ (Feature Name)	คำอธิบายและความสำคัญทางกายภาพ
0	Mean Accel Magnitude	ค่าเฉลี่ยขนาดความเร่งลัพธ์ (หน่วย g)
1	Max Accel Magnitude	ค่าสูงสุดขนาดความเร่ง (จุด Peak กระแทกพื้น)
2	Min Accel Magnitude	ค่าต่ำสุดขนาดความเร่ง (ช่วงสภาวะไร้น้ำหนัก Free-fall)
3	Std Accel Magnitude	ส่วนเบี่ยงเบนมาตรฐานของความเร่ง
4	Range Accel Magnitude	พิสัย (Max - Min) แสดงความแตกต่างของแรงกระแทก
5	Free-fall Ratio	สัดส่วนแซมเปิลที่มีความเร่ง $< 0.3g$ (สภาวะตกอิสระ)
6	Impact Ratio	สัดส่วนแซมเปิลที่มีความเร่ง $> 3.5g$

(แรงกระแทกพื้น)		
7	Mean Gyro Magnitude	ค่าเฉลี่ยความเร็วเชิงมุมลัพท์ (rad/s)
8	Max Gyro Magnitude	ค่าสูงสุดความเร็วเชิงมุม (การหมุนตัวเร็วขณะล้ม)
9	Std Gyro Magnitude	ส่วนเบี่ยงเบนมาตรฐานของความเร็วเชิงมุม
10	Signal Magnitude Area (SMA)	ผลรวมขนาดความเร่ง 3 แกนเฉลี่ย
11	Mean Horizontal Accel	ค่าเฉลี่ยความเร่งในระนาบแนวนอน (Ax, Az)
12	Max Horizontal Accel	ค่าสูงสุดความเร่งในระนาบแนวนอน
13	Mean Jerk	ค่าเฉลี่ยอัตราการเปลี่ยนความเร่ง ($ dAccel/dt $)
14	Max Jerk	ค่าสูงสุดอัตราการเปลี่ยนความเร่ง (กระแทกฉับพลัน)
15	Zero-Crossing Rate	อัตราการตัดผ่านระดับแรงโน้มถ่วง 1.0g
16	Correlation (Ax, Ay)	ค่าสหสัมพันธ์ระหว่างแกน X และ Y
17	Correlation (Ay, Az)	ค่าสหสัมพันธ์ระหว่างแกน Y และ Z

5. สถาปัตยกรรมโมเดลและไฮเปอร์พารามิเตอร์ (Model Architecture)

โมเดลถูกออกแบบเป็น Multi-Layer Perceptron (MLP) แบบ Shallow Dense Network

ที่คำนวณได้อย่างรวดเร็วและใช้หน่วยความจำ RAM ของ ESP32 น้อยกว่า 10 KB:

- **Input Layer:** 18 โหนด (เวกเตอร์คุณลักษณะ 18 Features)
- **Hidden Layer 1:** Dense 32 โหนด + Activation Function: ReLU
- **Hidden Layer 2:** Dense 16 โหนด + Activation Function: ReLU
- **Output Layer:** Dense 1 โหนด + Activation Function: Sigmoid (Output ความน่าจะเป็น 0.0 - 1.0)
- **Normalization:** ปรับสเกลข้อมูลด้วย StandardScaler (Mean และ Scale 18 มิติ)

- **Training Params:** Optimizer: Adam, Learning Rate: 0.001, Max Iterations: 500, Regularization (alpha): 0.0001

6. คำสั่งรันการเทรนโมเดลทีละขั้นตอน (Execution Commands)

เปิด Terminal / PowerShell แล้วรันคำสั่งด้านล่างนี้เพื่อเริ่มต้นกระบวนการเทรน:

```
python "c:\users\earth\desktop\pro\hardware1\tinym1\train_model.py"
```

ขั้นตอนการทำงานของสคริปต์: ลำดับเหตุการณ์ที่จะแสดงบน Terminal ระหว่างการเทรน:

- [1/4] โหลดข้อมูลการล้ม: โหลดไฟล์ CSV 52 ไฟล์จาก CollectData_Web สกัดได้ 155 หน้าต่าง Fall (Label = 1) และ 52 หน้าต่าง Normal (Label = 0)
- [2/4] โหลดข้อมูลกิจวัตร: โหลดไฟล์ ADL จาก SisFall สกัดได้ 279 หน้าต่าง Normal (Label = 0)
- [3/4] สรุปชุดข้อมูล: รวมตัวอย่างทั้งหมด 486 หน้าต่าง (Fall: 155, ADL: 331) และแบ่ง Train/Test 80:20 (Train 388, Test 98)
- [4/4] เทรนโมเดล: ปรับสเกล StandardScaler และเทรนโครงข่าย MLP

7. ผลการประเมินประสิทธิภาพของโมเดล (Evaluation Results)

เมตริกการประเมิน (Metric)	ผลลัพธ์ที่ได้ (Score)	คำอธิบายทางสถิติ
Test Accuracy (ความแม่นยำรวม)	98.98%	ความถูกต้องในการจำแนกบนชุดข้อมูลทดสอบ (Test Set 98 ตัวอย่าง)
Sensitivity / Recall (อัตราตรวจพบการล้ม)	100.00%	ตรวจพบการล้มได้ถูกต้องครบ 100% (TP=31, FN=0 ไม่มีเหตุการณ์ล้มหลุดรอด)
Specificity (อัตราจำแนกท่าปกติ)	98.51%	จำแนกกิจวัตรประจำวันถูกต้อง ไม่ส่งสัญญาณเตือนผิดพลาด (TN=66, FP=1)
F1-Score (Fall Class)	0.9841	ค่าเฉลี่ยฮาร์โมนิกระหว่าง Precision และ Recall ของคลาสการล้ม

8. การส่งออกเป็นไฟล์ C Header (fall_model.h)

สคริปต์จะทำการแปลงค่าน้ำหนัก Weights, Biases และพารามิเตอร์ Mean, Scale ของ StandardScaler ให้อยู่ในรูปของ C Arrays ค่าคงที่ (Constants) และบันทึกลงไฟล์ fall_model.h อัตโนมัติใน 3 ตำแหน่ง:

- ตำแหน่งที่ 1: c:\Users\Earth\Desktop\Pro\Hardware1\src\fall_model.h (สำหรับโปรเจกต์เฟิร์มแวร์หลัก)
- ตำแหน่งที่ 2: c:\Users\Earth\Desktop\Pro\CollectData_HardwareV2\src\fall_model.h (สำหรับโปรเจกต์เก็บข้อมูล)
- ตำแหน่งที่ 3: D:\ssPro\fall_model.h (สำหรับสำรองไฟล์โมเดลส่วนกลาง)

9. ขั้นตอนการคอมไพล์และแฟลชลงบอร์ด ESP32 (Firmware Deployment)

หลังจากได้ไฟล์ fall_model.h เรียบร้อยแล้ว ให้ทำการ Compile และ Upload เฟิร์มแวร์ลงบอร์ด ESP32 ผ่าน PlatformIO CLI:

```
# ไปยังโฟลเดอร์เฟิร์มแวร์ Hardware1
cd "c:\Users\Earth\Desktop\Pro\Hardware1"

# คอมไพล์และอัปโหลดลง ESP32
pio run --target upload
```

สถานะความพร้อม: เมื่ออัปโหลดเสร็จสิ้น บอร์ด ESP32 จะพร้อมตรวจจับการล้มแบบ Hybrid Real-Time TinyML ทันที!